SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 12

Name of Student : K.Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)
➢ Task: Use AI assistance to generate a Python program that implements a Stack data structure.
Instructions: Prompt AI to create a Stack class with the following methods:
➢ push(element), pop(), peek(), is_empty()
➢ Ensure proper error handling for stack underflow.
➢ Ask AI to include clear docstrings for each method.
Expected Output:
➢ A functional Python program implementing a Stack using a class.
➢ Properly documented methods with docstrings.

Prompt:-
Create a Python Stack class with standard operations and proper handling of overflow and underflow, including clear docstrings.

Code:-

SR UNIVERSITY

Assignment12.py > ...

```python
#Create a Python Stack class with standard operations and proper handling of overflow and underflow, includi
class Stack:
    """
    A Stack class that implements standard stack operations: push, pop, peek, and is_empty.
    The stack has a fixed capacity to handle overflow and underflow situations.
    """

    def __init__(self, capacity):
        """
        Initializes the stack with a given capacity.

        :param capacity: The maximum number of elements the stack can hold.
        """
        self.capacity = capacity
        self.stack = []

    def push(self, item):
        """
        Adds an item to the top of the stack. Raises an exception if the stack is full.

        :param item: The item to be added to the stack.
        :raises OverflowError: If the stack is full.
        """
        if len(self.stack) >= self.capacity:
            raise OverflowError("Stack overflow: Cannot add more items, the stack is full.")
        self.stack.append(item)

    def pop(self):
        """
        Removes and returns the item at the top of the stack. Raises an exception if the stack is empty.

        :return: The item at the top of the stack.
        :raises IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot remove items from an empty stack.")
        return self.stack.pop()

    def peek(self):
        """
```

```python
        Returns the item at the top of the stack without removing it. Raises an exception if

        :return: The item at the top of the stack.
        :raises IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek items from an empty stack.")
        return self.stack[-1]

    def is_empty(self):
        """
        Checks if the stack is empty.

        :return: True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Test cases
stack = Stack(3)
stack.push(1)
stack.push(2)
stack.push(3)
print(stack.peek())  # Output: 3
print(stack.pop())   # Output: 3
print(stack.pop())   # Output: 2
print(stack.is_empty())  # Output: False
print(stack.pop())   # Output: 1
print(stack.is_empty())  # Output: True
try:
    stack.pop()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Stack underflow: Cannot remove items from an empty stack.
try:
    stack.peek()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Stack underflow: Cannot peek items from an empty stack.
try:
    stack.push(4)
    stack.push(5)
    stack.push(6)
    stack.push(7)  # This should raise an error
except OverflowError as e:
    print(e)  # Output: Stack overflow: Cannot add more items, the stack is full.
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users/Dell/AppData/
3
3
2
False
1
True
Stack underflow: Cannot remove items from an empty stack.
Stack underflow: Cannot peek items from an empty stack.
Stack overflow: Cannot add more items, the stack is full.
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:-
The program correctly implements all stack operations with error handling and constant-time performance for core methods.

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)
➢ Task: Use AI to implement and compare Linear Search and Binary Search algorithms in Python.
Instructions:
➢ Prompt AI to generate:
➢ linear_search(arr, target)
➢ binary_search(arr, target)
➢ Include docstrings explaining:
➢ Working principle
➢ Test both algorithms using different input sizes.
Expected Output:
• Python implementations of both search algorithms.
• AI-generated comments and complexity analysis.
• Test results showing correctness and comparison.

Prompt:-
Write a Python program to implement and compare Linear Search and Binary Search with execution time analysis and docstrings.

Code:-

```python
ssignment12.py > ...
#task 2
#Write a Python program to implement and compare Linear Search and Binary Search with execution time analysis and
import time
def linear_search(arr, target):
    """
    Performs a linear search for the target in the given array.

    :param arr: The list of elements to search through.
    :param target: The element to search for.
    :return: The index of the target if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
def binary_search(arr, target):
    """Performs a binary search for the target in the given sorted array.
    :param arr: The sorted list of elements to search through.
    :param target: The element to search for.
    :return: The index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Test cases
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target = 5
# Linear Search
start_time = time.time()
result_linear = linear_search(arr, target)
end_time = time.time()
print(f"Linear Search: Target found at index {result_linear}, Time taken: {end_time - start_time:.6f} seconds")
# Binary Search
start_time = time.time()
result_binary = binary_search(arr, target)
end_time = time.time()
print(f"Binary Search: Target found at index {result_binary}, Time taken: {end_time - start_time:.6f} seconds")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users/Dell/AppData/Local/Progra
Linear Search: Target found at index 4, Time taken: 0.000000 seconds
Binary Search: Target found at index 4, Time taken: 0.000000 seconds
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:-

The program compares both algorithms effectively, showing linear search as O(n) and binary search as O(log n)

Task Description -3 (Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to confirm success.

Expected Output:

➢ Separate test file and implementation file.

➢ Test cases executed before implementation.

➢ **Final implementation passing all test cases.**

Prompt:-
Generate unit test cases for addition and subtraction without implementing the calculator to follow TDD principles.
Code:-

```
#Task 3
#Generate unit test cases for addition and subtraction without impl
import unittest

# 1. Empty Implementation (The "Red" phase of TDD)
class Calculator:
    def add(self, a, b):
        pass # Not implemented yet

    def subtract(self, a, b):
        pass # Not implemented yet

# 2. Test Cases
class TestCalculator(unittest.TestCase):
    def setUp(self):
        self.calc = Calculator()

    def test_add(self):
        self.assertEqual(self.calc.add(5, 3), 8)
        self.assertEqual(self.calc.add(-1, 1), 0)

    def test_subtract(self):
        self.assertEqual(self.calc.subtract(10, 5), 5)
        self.assertEqual(self.calc.subtract(0, 5), -5)

if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

Output:-

```
----------------------------------------
Ran 2 tests in 0.002s

FAILED (failures=2)
PS C:\Users\Dell\Desktop\PYTHON>
```

Justification:-

The tests are designed to fail initially, ensuring the final implementation is driven by test requirements.

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

➢ Task:

Use AI assistance to generate a Python program that implements a Queue datastructure.

Instructions: Prompt AI to create a Queue class with the following methods:

• enqueue(element), dequeue(), front(), is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate docstrings for all methods.

Expected Output:

➢ A fully functional Queue implementation in Python.

➢ Proper error handling and documentation.

Prompt:-

Create a Python Queue class with basic operations, error handling for overflow and underflow, and proper docstrings.

Code:-

SR UNIVERSITY

```python
#task 4
#Create a Python Queue class with basic operations, error handling for overflow and underflow, and
class Queue:
    """
    A Queue class that implements standard queue operations: enqueue, dequeue, peek, and is_empty.
    The queue has a fixed capacity to handle overflow and underflow situations.
    """

    def __init__(self, capacity):
        """
        Initializes the queue with a given capacity.

        :param capacity: The maximum number of elements the queue can hold.
        """
        self.capacity = capacity
        self.queue = []

    def enqueue(self, item):
        """
        Adds an item to the end of the queue. Raises an exception if the queue is full.

        :param item: The item to be added to the queue.
        :raises OverflowError: If the queue is full.
        """
        if len(self.queue) >= self.capacity:
            raise OverflowError("Queue overflow: Cannot add more items, the queue is full.")
        self.queue.append(item)

    def dequeue(self):
        """
        Removes and returns the item at the front of the queue. Raises an exception if the queue is

        :return: The item at the front of the queue.
        :raises IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot remove items from an empty queue.")
        return self.queue.pop(0)

    def peek(self):
        """
```

```python
    def peek(self):
        Returns the item at the front of the queue without removing it. Raises an except

        :return: The item at the front of the queue.
        :raises IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot peek items from an empty queue.")
        return self.queue[0]

    def is_empty(self):
        """
        Checks if the queue is empty.

        :return: True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0
# Test cases
queue = Queue(3)
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
print(queue.peek())  # Output: 1
print(queue.dequeue())   # Output: 1
print(queue.dequeue())   # Output: 2
print(queue.is_empty())  # Output: False
print(queue.dequeue())   # Output: 3
print(queue.is_empty())  # Output: True
try:
    queue.dequeue()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Queue underflow: Cannot remove items from an empty queue.
try:
    queue.peek()  # This should raise an error
except IndexError as e:
    print(e)  # Output: Queue underflow: Cannot peek items from an empty queue.
try:
    queue.enqueue(4)
    queue.enqueue(5)
    queue.enqueue(6)
    queue.enqueue(7)  # This should raise an error
except OverflowError as e:
    print(e)  # Output: Queue overflow: Cannot add more items, the queue is full.
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users/Dell/AppData/
1
1
2
False
3
True
Queue underflow: Cannot remove items from an empty queue.
Queue underflow: Cannot peek items from an empty queue.
Queue overflow: Cannot add more items, the queue is full.
```

Justification:-
The queue implementation handles all operations efficiently with O(1) time complexity and safe error handling.

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)
➢ Task:
Use AI to implement Bubble Sort & Selection Sort algorithm & compare their behavior.
Instructions:
• bubble_sort(arr), • selection_sort(arr)
➢ Include comments explaining each step.
➢ Add docstrings mentioning time and space complexity.
Expected Output:
• Correct Python implementations of both sorting algorithms.
• Complexity analysis in docstrings.

Prompt:-
Implement Bubble Sort and Selection Sort in Python and compare their performance with different input sizes.

Code:-

SR UNIVERSITY

```python
#task 5
#Implement Bubble Sort and Selection Sort in Python and compare their performance with d
class Queue:
    def __init__(self, max_size):

        self.items = []
        self.max_size = max_size

    def is_empty(self):

        return len(self.items) == 0

    def is_full(self):

        return len(self.items) == self.max_size

    def enqueue(self, element):

        if self.is_full():
            raise OverflowError("Queue Overflow: Cannot enqueue, the queue is full.")
        self.items.append(element)
        print(f"Enqueued: {element}")

    def dequeue(self):

        if self.is_empty():
            raise IndexError("Queue Underflow: Cannot dequeue from an empty queue.")
        # Pop index 0 to remove the first element added
        dequeued_item = self.items.pop(0)
        print(f"Dequeued: {dequeued_item}")
        return dequeued_item

    def front(self):

        if self.is_empty():
            raise IndexError("Queue is empty. No front element.")
        return self.items[0]

# --- Testing the Queue ---
if __name__ == "__main__":
    # Create a small queue of size 2 to easily test overflow
    q = Queue(max_size=2)

    q.enqueue("Customer 1")
    q.enqueue("Customer 2")
```

```python
print(f"Front of queue: {q.front()}")

# Trigger Overflow
try:
    q.enqueue("Customer 3")
except OverflowError as e:
    print(f"Error caught: {e}")

q.dequeue()
q.dequeue()

# Trigger Underflow
try:
    q.dequeue()
except IndexError as e:
    print(f"Error caught: {e}")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users/Dell/AppData/Local/Prog
Enqueued: Customer 1
Enqueued: Customer 2
Front of queue: Customer 1
Error caught: Queue Overflow: Cannot enqueue, the queue is full.
Dequeued: Customer 1
Dequeued: Customer 2
Error caught: Queue Underflow: Cannot dequeue from an empty queue.
```

Justification:-

Both sorting algorithms are correctly implemented, handle edge cases, and demonstrate $O(n^2)$ time complexity.